Module 1: Basics of Parallel Programming in Python (CS2)

Author: Xiaoyuan Suo, Webster University
Target Course: CS2 (Python-based), at Webster it's COSC 2110—Python
Prerequisite: students with prior OOP experience (e.g., C++ or Java)

This module introduces the foundational concepts of Parallel and Distributed Computing (PDC) using Python. It is designed as an entry point for undergraduate students who already understand object-oriented programming, loops, functions, and classes, but have little or no prior exposure to concurrency or parallelism.

The module combines conceptual grounding, hands-on programming, and performance reasoning, preparing students for more advanced parallel and performance-oriented coursework. I usually use this module in week 8 of this course.

1. Definitions and Core Concepts

This module introduces the following PDC concepts at a depth sufficient for instructors to adapt the material to different courses and levels:

Parallel vs. Distributed Computing

- Parallel computing: multiple execution units (threads or processes) working concurrently on a single machine with shared or logically shared resources.
- Distributed computing: multiple machines cooperating via message passing, each with private memory.

Students explore *why parallelism does not always lead to linear speedup* using intuitive examples (e.g., shoveling snow, planting trees) before transitioning to code-level demonstrations

(see slides)

Threads and Processes

- Threads share memory space and are lightweight but subject to synchronization issues.
- Processes have separate memory spaces, avoiding some data races but introducing communication overhead.

Python-specific realities are emphasized:

- The Global Interpreter Lock (GIL) limits CPU-bound threading performance.
- multiprocessing is the primary tool for CPU-bound parallelism in Python.

Race Conditions and Synchronization

Students are introduced to:

- Race conditions caused by unsynchronized access to shared data.
- Locks, RLocks, and critical sections.
- Deadlock conditions and why they occur.

These concepts are illustrated through small, intentionally flawed examples that demonstrate nondeterministic behavior, followed by corrected versions using locks.

Parallel Decomposition and Reduction

- Task decomposition (splitting work across workers).
- Reduction patterns (e.g., summation).
- Two-level reduction as a scalable design pattern.

These ideas are reinforced through the parallel sum assignment, which requires students to reason about decomposition, correctness, and performance.

(see assignment)

2. Course Integration: Where This Module Fits

Suitable Courses

- CS2 (Python-based)
- Data Structures (conceptual parallelism)
- Systems Programming (threads, processes, locks)
- Data Science / Analytics
- Introduction to HPC or Performance Engineering

Integration with Existing Topics

Existing Topic	PDC Integration
Loops	Data parallel loops
Functions	Task-based parallelism
Classes	Thread/process encapsulation
Arrays	Parallel reduction
Algorithm analysis	Speedup & efficiency

Parallelism is introduced as a natural extension of algorithmic thinking rather than a separate specialty topic.

3. Downstream Courses and Concepts Supported

This module can support later instruction in:

- Performance engineering
- Parallel algorithms
- Operating systems
- Scientific computing
- Machine learning (data parallelism)
- HPC cluster usage

The parallel reduction assignment serves as a conceptual bridge between:

- Sequential algorithm correctness
- Parallel correctness
- Real-world performance measurement on shared systems .

4. Limitations

This module intentionally highlights common misconceptions:

- "More cores = faster" is often false due to overhead.
- Python threads do not scale for CPU-bound workloads (GIL).
- Poor decomposition can be slower than sequential code.
- Synchronization introduces serialization.
- Memory bandwidth becomes a bottleneck before CPU saturation.

Students encounter these limitations empirically through required performance plots and controlled experiments, see Assignment included.

Limitations of this module:

- There is no required textbook for this module
- Assignment can be easily done by AI, so I require students to do a part of it in class
- Problems with using Python to introduce PDC:
    - Limited true parallelism**:** Python's Global Interpreter Lock (GIL) prevents CPU-bound threads from running in parallel, and interpreter overhead further limits scalability; C++ supports true multithreading with low overhead.
    - Poor hardware-level control: Python offers limited control over memory layout, cache behavior, and NUMA effects, while C++ allows fine-grained management critical for high-performance parallel computing.

Reinforce that Python is a gateway language, not the end goal.

5. Included Materials

- Lecture slides: *Parallel Computing with Python*

Python parallel

- Assignment: *Parallel find_sum with Performance Evidence*

  Assignment12_parallel_sum

- Sample code: threading, multiprocessing, locks
- Performance plotting requirements